

MANUEL

Manuel d'utilisation — CodeScan

Analyseur statique de code **sans API d'IA**, basé sur des règles AST et regex et sur la base de vulnérabilités OSV.dev.

Sommaire

1. [À propos](#1--à-propos)
 1. [Prérequis et installation](#2--prérequis-et-installation)
 1. [Démarrage rapide](#3--démarrage-rapide)
 1. [Référence des options CLI](#4--référence-des-options-cli)
 1. [Exemples d'utilisation détaillés](#5--exemples-dutilisation-détaillés)
 1. [Fonctionnement interne (pipeline)](#6--fonctionnement-interne-pipeline)
 1. [Les rapports de sortie](#7--les-rapports-de-sortie)
 1. [Codes de sortie](#8--codes-de-sortie)
 1. [Référence des règles](#9--référence-des-règles)
 1. [Personnalisation des règles](#10--personnalisation-des-règles)
 1. [Exécution des tests](#11--exécution-des-tests)
 1. [Dépannage et FAQ](#12--dépannage-et-faq)
 1. [Intégration CI/CD](#13--intégration-cicd)
 1. [Limites connues](#14--limites-connues)
 1. [Mode IRON MAN AI — audit Kali (sites web)](#15-mode-iron-man-ai-audit-kali-sites-web)
-

1. À propos

CodeScan parcourt un projet (dossier local ou dépôt GitHub cloné) et repère :

- les **failles de sécurité** classiques : injection SQL, XSS, exécution de code/commandes, désérialisation non sûre, `except: nu...` ;
- les **secrets exposés** : clés AWS/Google/Stripe, tokens GitHub/Slack/npm, clés privées, mots de passe en clair — par patterns regex **et** par calcul d'**entropie de Shannon** ;
- les **failles de configuration** : CORS permissif, regex dynamique utilisateur (ReDoS), superglobales PHP lues directement, `SELECT *` ;
- la **dette technique et qualité** : TODO/FIXME, fonctions trop longues/complexes (cyclomatiques **et cognitives**), imbrication excessive, trop de paramètres, `==` au lieu de `===`, blocs vides, Nombres magiques, code commenté, `console.log` de débogage, copie profonde via JSON ;
- la **performance/asynchrone (JS)** : I/O synchrones bloquantes dans des gestionnaires, I/O successives en boucle (N+1), boucles quadratiques $O(n^2)$, copie profonde coûteuse ;
- les **dépendances vulnérables** : croisement de `requirements.txt`, `package.json`, `composer.json` ... avec la base de CVE **OSV.dev** ;
- une **note de qualité /100 + lettre (A→F)** qui synthétise tout le projet.

Contrairement à un assistant IA, CodeScan ne « devine » rien : chaque résultat provient d'une règle déterministe (AST Python ou regex) et est associé à une sévérité, une description et une recommandation.

2. Prérequis et installation

Prérequis	Rôle
Python ≥ 3.10	Interpréteur (le cœur utilise uniquement la bibliothèque standard)
git (optionnel)	Nécessaire uniquement pour le mode <code>--repo</code> (clonage)
requests (optionnel)	Nécessaire uniquement pour la vérification OSV.dev

```
# Depuis le dossier codescan/  
pip install -r requirements.txt      # installe requests (facultatif)
```

Astuce Windows : si `python` n'est pas reconnu, utilisez le lanceur

`py` à la place (ex. `py main.py --path ./mon_projet`).

Vérifier l'installation :

```
py main.py --version      # affiche la version
py main.py --help         # affiche l'aide complète
```

3. Démarrage rapide

```
# 1. Analyser un dossier local (résumé console uniquement)
py main.py --path ./mon_projet

# 2. Analyser un dossier et générer un rapport HTML
py main.py --path ./mon_projet --output report.html

# 3. Analyser un dépôt GitHub (cloné puis supprimé automatiquement)
py main.py --repo https://github.com/user/repo --output report.json

# 4. Ne garder que les failles haute/critique
py main.py --path ./mon_projet --output report.html --severity-min=high

# 5. Démo sur le projet d'exemple fourni
py main.py --path exemples --output report.html --verbose

# 6. Rapport « épuré » : sans qualité de code ni note
py main.py --path ./mon_projet --no-quality --no-score --output report.html
```

Le résultat s'affiche immédiatement dans le terminal ; le rapport (HTML ou JSON) est écrit dans le fichier indiqué par `--output` .

4. Référence des options CLI

```
usage: CodeScan [-h] [--path CHEMIN] [--repo URL] [-o FICHIER]
                [--severity-min {critical,high,medium,low}]
                [--no-deps] [--no-quality] [--no-score]
                [--rules FICHIER] [-v] [--version]
```

Option	Description	Défaut
<code>--path</code> <code><CHEMIN></code>	Dossier local du projet à analyser.	—
<code>--repo</code> <code><URL></code>	URL GitHub à cloner puis analyser (<code>https://github.com/user/repo</code>).	—
<code>-o, --output</code> <code><FICHIER></code>	Rapport de sortie. Extension <code>.json</code> ou <code>.html</code> (le format est déduit de l'extension). Sans cette option, résumé console seul.	—
<code>--severity-min</code> <code><NIVEAU></code>	Sévérité minimale des résultats retenus. Les niveaux inférieurs sont filtrés du rapport.	<code>low</code>
<code>--no-deps</code>	Désactive l'interrogation réseau d'OSV.dev (analyse hors ligne).	<code>false</code>
<code>--no-quality</code>	Désactive l'analyseur de qualité de code (métriques de fonction, lignes longues, async/perf JS, égalité faible...). Les règles AST Python et regex de sécurité restent actives.	<code>false</code>
<code>--no-score</code>	Ne calcule ni n'affiche la note /100 ; la clé <code>score</code> disparaît des rapports JSON/HTML.	<code>false</code>
<code>--rules</code> <code><FICHIER></code>	Chemin d'une base de patterns alternative au format JSON (voir §10).	<code>rules/patterns.json</code>
<code>--exclude</code> <code><MOTIF></code>	Exclut les fichiers dont le chemin relatif contient le motif (option répétable ou motifs séparés par des virgules ; jokers <code>*</code> / <code>?</code> acceptés). Ex. <code>--exclude tests,migrations</code> ou <code>--exclude '*.min.js'</code> .	—
<code>-v, --verbose</code>	Affiche le détail : fichiers explorés, messages des analyseurs, liste complète des findings.	<code>false</code>
<code>--version</code>	Affiche la version et quitte.	—
<code>-h, --help</code>	Affiche l'aide et quitte.	—

Contraintes d'usage :

- `--path` et `--repo` sont **mutuellement exclusifs** mais l'un des deux est **obligatoire** ; sinon CodeScan renvoie une erreur (code 2).
- `--output` doit se terminer par `.json` ou `.html` ; toute autre extension produit une erreur (code 2).

5. Exemples d'utilisation détaillés

5.1 Analyse locale avec rapport HTML

```
py main.py --path ./mon_projet --output rapport.html --severity-min=medium --verbose
```

- Analyse tous les fichiers du projet (hors dossiers ignorés et binaires).
- Écrit `rapport.html` (autonome, consultable hors ligne).
- `--verbose` liste chaque fichier exploré dans la console.

5.2 Analyse d'un dépôt GitHub

```
py main.py --repo https://github.com/django/django --output scan.json --no-deps
```

- Clone le dépôt en profondeur 1 dans un dossier temporaire.
- Analyse puis **supprime** automatiquement le clone (rien ne reste sur disque).
- `--no-deps` évite ici la (longue) requête OSV.dev sur les centaines de dépendances d'un gros projet.

5.3 Filtrage par sévérité (pour un tri rapide)

```
py main.py --path ./mon_projet --output urgent.html --severity-min=critical
```

Ne conserve que les failles **critiques**. Utile pour le tri des incidents.

5.4 Mode hors ligne

```
py main.py --path ./mon_projet --no-deps
```

Ne fait **aucun** accès réseau : les CVE de dépendances ne sont pas vérifiées. Le reste de l'analyse est inchangé.

5.5 Base de règles personnalisée

```
py main.py --path ./mon_projet --rules ./ma-base.json
```

Charge les patterns d'un fichier JSON alternatif (voir §10).

5.6 Exclure des fichiers/dossiers de l'analyse

```
py main.py --path ./mon_projet --exclude tests,migrations --exclude '*.min.js'
```

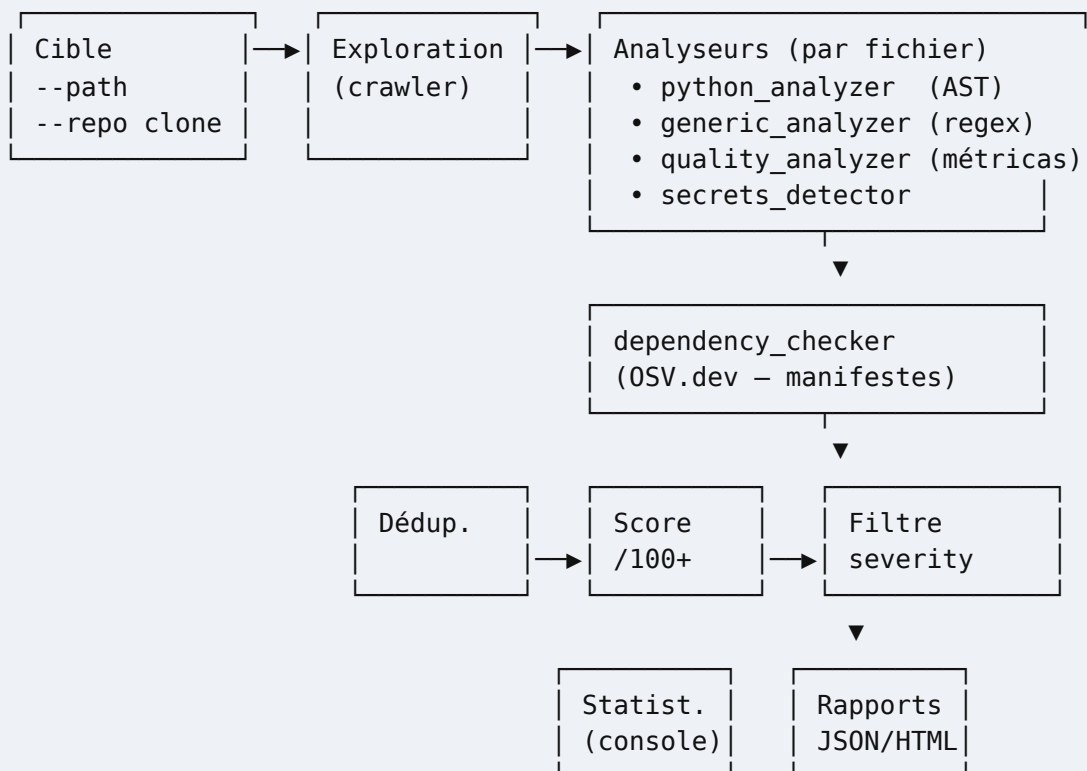
Exclut tout fichier dont le chemin relatif contient `tests` ou `migrations`, ainsi que les fichiers `*.min.js` (jokers acceptés). Pratique pour ignorer du code généré, des fixtures de test ou des bundles.

5.7 Rapport « sécurité uniquement »

```
py main.py --path ./mon_projet --no-quality --no-score --output scan.json
```

Désactive l'analyse de qualité (métriques de fonction, perf JS...) **et** le calcul de la note : seuls les findings de sécurité/dépendances/secrets restent, et le rapport JSON n'a pas de clé `score`. Utile si vous ne voulez pas de bruit qualité ou si vous préférez un score calculé par un autre outil.

6. Fonctionnement interne (pipeline)



Étapes détaillées :

1. **Récupération de la cible** — dossier local ou clonage `git clone --depth 1`.
1. **Exploration** (`scanner/crawler.py`) — parcours récursif ; ignore les dossiers `node_modules/`, `.git/`, `venv/`, `__pycache__/`, `dist/` ..., les extensions binaires, et les fichiers > 2 Mo. Chaque fichier est classé : `python`, `source`, `config`, `manifest`.
1. **Analyse Python** (`scanner/python_analyzer.py`) — parsing AST : `eval`, `exec`, `pickle/marshal`, `except: nu`, `subprocess(shell=True)`, injection SQL, `assert` de sécurité, imports dangereux, secrets nommés, et complexité de code : longueur de fonction, cyclomatique **et cognitive**, imbrication profonde, trop de paramètres, `else` après `return`, comparaison booléenne « redondante », blocs `except` vides.
1. **Analyse générique** (`scanner/generic_analyzer.py`) — regex sur le contenu : SQL, XSS, commandes, TODO/FIXME, CORS permissif, regex dynamique (ReDoS), superglobales PHP, `SELECT *` ...
1. **Analyse de qualité** (`scanner/quality_analyzer.py`, désactivée par `--no-quality`) — métriques de fonctions sur les langages à accolades (JS/TS/PHP/Java/C#/Go/...) par équilibrage d'accolades sur un code dont les chaînes/templates/commentaires sont masqués ; règles lignes : lignes > 120, fichiers > 500 lignes, code commenté, nombres magiques, `==` vs `===` ; règles de débogage JS (`console.log`, `alert`, copie profonde JSON) ; règles **performance/asynchrone** : I/O synchrones bloquantes (hors fonctions `config/init/load` et top-level), I/O successives dans les boucles (N+1, hors `Promise.all` / `for await`), boucles O(n²) avec `.includes` / `indexOf` (hors caches `Map` / `Set`).
1. **Détection de secrets** (`scanner/secrets_detector.py`) — patterns forts, **entropie de Shannon** (seuil 4,5 bits/caractère), mots de passe en clair dans les fichiers de configuration (test appliqué à la clé seule, pas à la ligne entière). Les fichiers de verrouillage générés automatiquement (`package-lock.json`, `yarn.lock`, `pnpm-lock.yaml` ...) sont ignorés.
1. **Vérification des dépendances** (`scanner/dependency_checker.py`) — parsing des manifestes puis requêtes **batch** vers `api.osv.dev`.
1. **Post-traitement** — **déduplication**, calcul du **score /100 + grade** sur l'ensemble déduplicé (indépendant de `--severity-min`), filtrage de l'affichage, tri, statistiques, rapport.

7. Les rapports de sortie

7.1 Résumé console

Affiché systématiquement :

```
CodeScan – Résumé de l'analyse

Cible          : .\mon_projet
Fichiers analysés : 42
Score de qualité  : 61/100 (C)
Total findings   : 12

Par sévérité :
critical 2
high     5
medium   3
low      2

Par catégorie :
injection 4
secrets   3
...
```

Les couleurs ANSI sont activées automatiquement si le système est un TTY (désactivées si la sortie est redirigée, pour un pipeline). La ligne

Score de qualité n'apparaît que si le score est calculé (par défaut) ;

avec `--no-score` elle devient `Score de qualité : désactivé (--no-score)`.

7.2 Rapport JSON

Structure exacte (compatible avec des outils tiers ou du CI) :


```

{
  "meta": {
    "tool": "CodeScan",
    "version": "1.1.0",
    "timestamp": "2026-08-04T14:05:00",
    "target": "D:\\projet",
    "severity_min": "low"
  },
  "target": "D:\\projet",
  "summary": {
    "total_findings": 12,
    "files_scanned": 42,
    "by_severity": { "critical": 2, "high": 5, "medium": 3, "low": 2 },
    "by_category": { "injection": 4, "secrets": 3 }
  },
  "score": {
    "score": 61,
    "grade": "C",
    "total_findings": 12,
    "files_scanned": 42,
    "security": 6,
    "quality": 5,
    "performance": 1,
    "by_level": { "CRITIQUE": 2, "À REVOIR": 8, "MINEUR": 2 },
    "by_domain_pct": { "security": 50.0, "quality": 41.7, "performance": 8.3 },
    "weights": { "critical": 5, "high": 2, "medium": 1, "low": 0.5 }
  },
  "findings": [
    {
      "file": "app.py",
      "line": 30,
      "column": 0,
      "rule_id": "py-subprocess-shell",
      "category": "injection",
      "severity": "high",
      "title": "subprocess avec shell=True",
      "description": "subprocess avec shell=True invoque le shell système...",
      "recommendation": "Utiliser subprocess sans shell (liste d'arguments)...",
      "snippet": "subprocess.run(cmd, shell=True)",
      "language": "python",
      "cve": "",
      "source": "python_analyzer"
    }
  ]
}

```

7.3 Rapport HTML (structure Herald)

- **Autonome** : CSS embarqué, aucune ressource externe, consultable hors ligne.
- **Adaptatif** : thème sombre/clair selon les préférences du navigateur.

- **Imprimable en PDF** : `@media print` format **A4** (`@page { size: A4; margin: 12mm }`), cartes et relevés avec `break-inside: avoid` , `print-color-adjust: exact` — « Imprimer en PDF » dans le navigateur donne un document fidèle sans coupes.
- **Structure** (parité avec le rapport de référence Herald) : 1. **en-tête** : titre, badge « **Sans IA** », cible, version, date ; 2. **hero score** : grand chiffre `score/100` (ex. `61/100`), **lettre de grade colorée** (A→F) et barre de progression de la note ; 3. **7 cartes de synthèse** : Relevés, Critiques, À revoir, Fichiers, Sécurité, Qualité, Performance ; 4. **répartition par catégorie** avec barres proportionnelles ; 5. **findings groupés par niveau lisible** : `CRITIQUE` → `À REVOIR` → `MINEUR` (les 3 sections apparaissent toujours, même vides — parité Herald), chaque entrée avec **badge rule_id**, titre, badge de sévérité (`critical` rouge foncé, `high` orange, `medium` jaune, `low` bleu), **fichier:ligne**, description, **recommandation** et extrait de code en `<pre>` échappé.

Sans score (`--no-score`), le hero et les cartes disparaissent, mais le détail par niveau reste.

7.4 Score de qualité /100 et grade

Le score synthétise l'état global d'un projet en **une note /100** et une

lettre **A → F**. Il est calculé par `scanner/scorer.py` :

```
WEIGHTS      = {critical: 5, high: 2, medium: 1, low: 0.5}
densité      = Σ(poids[sev] × nb[sev]) / fichiers
crit_ratio   = nb(critical) / fichiers
raw          = 100 / (1 + densité/15.5) × (1 - 0.5 × crit_ratio)
score        = clamp(round(raw), 0, 100)
```

Points clés	Détail
Normalisé par taille	Divisé par le nombre de fichiers → comparable entre projets
Critiques lourds	Pénalité proportionnelle en plus : <code>critical</code> pèse 5 et fait tomber la note (pénalité × nb critiques)
Lettres	A ≥ 90 , B ≥ 80 , C ≥ 70 , D ≥ 40 , F < 40 (grille 5 crans, reproduit « 49/100 D » du rapport de référence)
Domaine	<code>security</code> (injection/xss/secrets/security_misc/dependencies), <code>quality</code> (code_quality), <code>performance</code>

Le score est calculé sur **l'ensemble des findings dédoublés, avant**

filtrage `--severity-min` : la note reflète l'état réel du projet, pas ce qui

est affiché dans le listing.

8. Codes de sortie

Code	Signification
0	Analyse terminée, aucune faille critique ou haute détectée.
1	Analyse terminée avec au moins une faille critique/haute , OU erreur d'exécution (dossier/repo introuvable, clonage échoué, règles illisibles, écriture impossible).
2	Erreur d' utilisation de la CLI (cible manquante, format <code>--output</code> invalide).

Le code 1 en présence de failles permet d'**interrompre un pipeline CI** (voir §13). Pour distinguer une vraie erreur d'une simple présence de failles, vérifiez la présence du message `[ERREUR]` dans la sortie stderr.

9. Référence des règles

9.1 Analyseur Python (AST) — fichier `.py`

ID	Sévérité	Détection
py-dangerous-eval	critical / high	eval() / exec() (critical si l'argument vient de input())
py-pickle-load	high	pickle.loads/load , marshal.loads/load , yaml.load
py-subprocess-shell	high	subprocess.*(..., shell=True)
py-sql-injection	high	requête SQL concaténée, f-string ou %
py-hardcoded-secret	high	variable nommée password / api_key / secret / token = littéral
py-os-system	high	os.system() / os.popen()
py-bare-except	medium	except: sans type
py-assert-security	medium	assert sur un contrôle de privilège (désactivé avec - 0)
py-dangerous-import	medium	import de pickle , marshal , shelve , telnetlib
py-long-function	low	fonction > 50 lignes
py-high-complexity	medium	complexité cyclomatique > 10
py-cognitive-complexity	medium	complexité cognitive > 15
py-deep-nesting	medium	imbrication > 4 niveaux
py-too-many-params	medium	plus de 4 paramètres
py-else-after-return	medium	bloc else après un return dans le if
py-redundant-boolean	low	x == True / x is False (préférer x)
py-empty-except	medium	bloc except sans aucune instruction
py-request-without-timeout	medium	requests.*() sans timeout (blocage possible, DoS)
py-verify-false	high	requests.*(..., verify=False) : vérification TLS désactivée
py-weak-hash	medium	hashlib.md5/sha1 (ou hashlib.new("md5"))
py-insecure-random	medium	random.* pour un jeton/clé/mot de passe (contexte sécurité)
py-xml-unsafe	high	parsing XML sans defusedxml importé (risque XXE/bombe XML)
py-tempfile-mktemp	medium	tempfile.mktemp() (nom prévisible, course)

9.2 Analyseur générique (regex) — multi-langages

ID	Langages	Sévérité	Détection
generic-sql-injection	php, js, ts, java, c#, ruby, go...	high	requête SQL construite dynamiquement
generic-xss-innerhtml	js, ts, html	high	<code>.innerHTML =</code>
generic-xss-dangerouslyset	js, ts	high	<code>dangerouslySetInnerHTML</code> (React)
generic-xss-document-write	js, ts	high	<code>document.write()</code>
generic-xss-vhtml	js, ts	medium	directive Vue <code>v-html</code>
generic-eval	js, ts, php	high	<code>eval()</code>
generic-php-unserialize	php	critical	<code>unserialize()</code>
generic-command-exec	php, ruby, bash	high	<code>shell_exec</code> , <code>system</code> , <code>passthru</code> , <code>popen</code> ...
generic-node-child-process	js, ts	high	<code>child_process.exec/spawn</code>
generic-node-child-process-import	js, ts	medium	import de <code>child_process</code>
generic-js-exec-call	js, ts	medium	appel <code>exec()</code> (à vérifier)
generic-java-runtime-exec	java, kotlin	high	<code>Runtime.exec</code> , <code>ProcessBuilder</code>
generic-secret-assignment	non-Python	high	secret affecté en dur
generic-todo-fixme	tous	low	<code>TODO</code> , <code>FIXME</code> , <code>HACK</code> , <code>XXX</code>
generic-cors-permissive	php, js, ts, python	high	<code>Access-Control-Allow-Origin: *</code>
generic-regex-dos	js, ts	high	<code>new RegExp(...)</code> sur valeur non-littérale (risque ReDoS)
generic-php-superglobal-input	php	medium	lecture brute de <code>\$_GET</code> / <code>\$_POST</code> / <code>\$_REQUEST</code>

generic-sql-select-star	sql, php, js	medium	requête <code>SELECT *</code> non ciblée
generic-weak-crypto	php, js, ts, java, c#, go, ruby...	medium	md5 / sha1 / DES / RC4 ... y compris <code>createHash("sha1")</code> et <code>MessageDigest.getInstance("MD5")</code>
generic-php-xxe	php	high	<code>simplexml_load_*</code> , <code>->loadXML</code> (risque XXE)
generic-path-traversal	php, js, ts, python, java...	medium	chemin de fichier construit par concaténation (<code>fopen("up/" . \$n)</code> , <code>readFileSync("d/" + f)</code>)
generic-php-insecure-rand	php	low	<code>rand()</code> / <code>mt_rand()</code> (préférer <code>random_int()</code>)

9.2b Analyseur de qualité (à accolades + Python)

Les IDs ci-dessous portent le préfixe du langage : `js-*` pour le

JavaScript, `ts-*` pour TypeScript, `php-*` , `java-*` , `csharp-*` , `go-*` ,

`kotlin-*` , `scala-*` , `rust-*` , `swift-*` , `c-*` , `cpp-*` ... (`py-*` sur

Python, via AST — voir §9.1). Seuils centralisés dans

`scanner/thresholds.py` .

Règle (préfixé)	Sév	Seuil	Détection
*-function-too-long	low	50 lignes	fonction trop longue
*-cognitive-complexity	medium	15	complexité cognitive élevée
*-high-complexity	medium	10	complexité cyclomatique élevée
*-deep-nesting	medium	4	imbrication excessive
*-too-many-params	medium	4	trop de paramètres
*-long-line	low	120	ligne trop longue
*-file-too-long	low	500	fichier trop long
*-redundant-boolean	low	—	<code>x == true</code> / <code>x === false</code>
*-loose-equality	low	—	<code>==</code> (préférer <code>===</code>)
*-else-after-return	medium	—	<code>else</code> après un <code>return</code>
*-empty-catch	medium	—	bloc <code>catch</code> vide (erreurs avalées)
*-commented-out-code	low	≥ 3 lignes	code commenté (probablement mort)
*-magic-number	low	—	nombre littéral non nommé (conservateur)
*-no-debug-console	low	—	<code>console.log/warn/error</code> (débogage)
*-no-alert	low	—	<code>alert</code> / <code>confirm</code> / <code>prompt</code>
*-deep-clone-json	medium	—	copie profonde via <code>JSON.parse(JSON.stringify(x))</code>
perf-blocking-sync-io	high	—	<code>readFileSync</code> / <code>execSync</code> ... bloquants (hors init/connexion)
perf-io-in-loop	medium	—	<code>await fetch/query</code> dans une boucle (N+1) hors <code>Promise.all</code>
perf-quadratic-loop	medium	—	<code>.includes()</code> / <code>indexOf</code> dans une boucle sur tableau ($O(n^2)$)

Les métriques de fonction JS/PHP/etc. sont calculées par équilibrage

d'accolades sur un code masqué (chaînes/commentaires/templates) : les

valeurs sont **indicatives**. Les anti-faux-positifs ignorent les appels

dans les fonctions `config / init / load`, `Promise.all`, `for await`, les

caches `Map / Set` et les littéraux de constantes.

9.3 Détecteur de secrets

ID	Sévérité	Détection
secret-aws-access-key	critical	AKIA... / ASIA... (20 car.)
secret-aws-secret-key	critical	<code>aws_secret_access_key = "...40 car..."</code>
secret-github-token	critical	<code>ghp_...</code> , <code>github_pat_...</code>
secret-stripe	critical	<code>sk_live_...</code> , <code>rk_live_...</code>
secret-private-key	critical	bloc <code>-----BEGIN ... PRIVATE KEY-----</code>
secret-google-api	high	AIza... (39 car.)
secret-slack-token	high	<code>xox[baprs]-...</code>
secret-npm-token	high	<code>npm_...</code>
secret-bearer	high	Bearer <jeton>
secret-slack-webhook	high	URL <code>hooks.slack.com/services/...</code>
secret-jwt	medium	JWT <code>eyJ...</code>
secret-high-entropy	low	chaîne ≥ 16 car. à forte entropie (Shannon $\geq 4,5$)
config-plaintext-secret	high	secret en clair dans <code>.env / .json / .yaml / .ini</code>

9.4 Dépendances

ID	Sévérité	Détection
dep-cve	de critical à low	CVE d'un paquet via OSV.dev (sévérité déduite du score CVSS ou du niveau GHSA)

Manifestes supportés : `requirements*.txt`, `package.json`, `composer.json`,

`Gemfile`, `go.mod`, `Pipfile`, `Cargo.toml`, `pom.xml`.

9.5 Catégories

`injection` · `secrets` · `xss` · `code_quality` · `security_misc` ·

performance · dependencies

Ces catégories sont regroupées en **domaines** de score : security

(injection, secrets, xss, security_misc, dependencies), quality

(code_quality), performance.

10. Personnalisation des règles

Toutes les règles **regex** vivent dans `rules/patterns.json` : vous pouvez en ajouter, modifier ou retirer **sans toucher au code**.

Schéma d'une entrée generic (source)

```
{
  "id": "generic-hash-faible",
  "category": "security_misc",
  "severity": "medium",
  "languages": ["php", "javascript"],
  "pattern": "(?i)\\b(md5|sha1)\\s*\\(",
  "title": "Fonction de hachage faible",
  "description": "md5/sha1 ne sont pas sûrs pour les mots de passe.",
  "recommendation": "Utiliser bcrypt, argon2 ou au minimum sha256 avec sel."
}
```

Schéma d'une entrée secrets

```
{
  "id": "secret-nouvelle-cle",
  "name": "Clé privée NouveauService",
  "category": "secrets",
  "severity": "critical",
  "pattern": "\\bNS_[A-Za-z0-9]{24}\\b",
  "title": "Clé NouveauService exposée",
  "description": "Une clé d'API NouveauService est visible dans le code.",
  "recommendation": "Révoquer la clé et utiliser l'environnement."
}
```

Champs disponibles

Champ	Règles <code>generic</code>	Règles <code>secrets</code>	Obligatoire
id	✓	✓	oui
category	✓	✓	oui
severity	✓	✓	oui
pattern (regex)	✓	✓	oui, sauf règles <code>builtin</code>
languages (liste)	✓	X (scan global)	non
title	✓	✓	oui
description	✓	✓	recommandé
recommendation	✓	✓	recommandé
builtin	✓	✓	non (<code>entropy</code> pour les secrets)

Règles `builtin` (calculées par le code)

Une seule règle reste « builtin » dans le stock : `entropy` (détection des chaînes à forte entropie dans la section `secrets`). Elle n'a pas de `pattern` mais un champ `builtin: "entropy"` .

Les métriques de fonction (longueur, complexité, imbrication...) ne sont **plus** des patterns : elles sont calculées directement par `scanner/python_analyzer.py` (AST, exact) et `scanner/quality_analyzer.py` (accolades, heuristique). Réglez leurs seuils dans `scanner/thresholds.py` (le JSON de règles ne les contient pas).

Validation : CodeScan ignore une règle dont la regex est invalide et

l'indique dans la console (`pattern invalide`), sans interrompre l'analyse.

11. Exécution des tests

```
cd codescan
python -m unittest discover tests -v
```

La suite (~**240 tests**) couvre :

- chaque règle de l'analyseur Python sur du code vulnérable **et** du code sain (anti-faux-positifs), y compris les nouvelles règles de qualité (complexité cognitive, imbrication, paramètres, `else` après `return` ...) ;
 - les règles regex génériques (SQL, XSS, eval, TODO, CORS, ReDoS...) ;
 - l'analyseur de qualité (métriques de fonction JS/PHP, lignes longues, async/perf, anti-faux-positifs `Promise.all` /config) ;
 - le détecteur de secrets (AWS, GitHub, Stripe, entropie, placeholders) ;
 - le crawler (dossiers ignorés, binaires, détection de langage) ;
 - le parsing des manifestes et le calcul des scores CVSS ;
 - le calcul du score /100 (calibration sur la référence Herald → 49/100 D, bandes A-F, domaines) et du rapport HTML (hero, cartes, niveaux, échappement, CSS print) ;
 - le filtrage par sévérité.
-

12. Dépannage et FAQ

« `python` n'est pas reconnu » (Windows)

Le lanceur `py` est installé avec Python : utilisez `py main.py ...`.

Sinon, désactivez l'alias « App execution alias » dans les paramètres Windows.

« **git clone a échoué : Repository not found** »

L'URL est invalide ou le dépôt est privé. CodeScan ne gère **pas**

l'authentification : utilisez un dépôt public ou clonez-le d'abord

manuellement puis analysez-le avec `--path`.

« **OSV.dev indisponible** »

Le réseau est coupé ou le service est en maintenance. CodeScan l'indique dans

la console et **continue** l'analyse sans la partie dépendances. Utilisez

`--no-deps` pour un fonctionnement hors ligne sans message d'erreur.

« **Le rapport affiche beaucoup de faux positifs** »

- Les heuristiques (`generic-long-function` , `generic-high-complexity` , `secret-high-entropy`) sont des **indicateurs**, pas des certitudes : vérifiez chaque résultat.
- Réduisez le bruit avec `--severity-min=medium` OU `--severity-min=high` .

« Un fichier n'est pas analysé »

Vérifiez qu'il n'est pas dans un dossier ignoré (`node_modules/` , `.git/` , `venv/` , `__pycache__/` ...), qu'il ne fait pas plus de 2 Mo et que son extension est reconnue (voir `LANG_BY_EXT` dans `scanner/crawler.py`).

« Les caractères accentués sont mal affichés dans la console »

L'affichage est forcé en UTF-8 par CodeScan. Si le terminal ne le supporte pas, définissez `PYTHONIOENCODING=utf-8` ou activez le terminal UTF-8.

« Comment supprimer les secrets de l'historique git ? »

CodeScan les détecte, mais il ne les efface pas. Après correction du code, régénérez l'historique (`git filter-repo`), **révoquez** les secrets exposés sur le service concerné, puis ajoutez les fichiers à `.gitignore` .

13. Intégration CI/CD

Exemple d'étape GitHub Actions :

```
- name: Analyser le code avec CodeScan
  run: |
    pip install -r requirements.txt
    python main.py --path . --output codescan.json --severity-min=high
    # Le workflow échoue si des failles haute/critique sont détectées
    # (code de sortie 1).
- name: Archiver le rapport
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: codescan-report
    path: codescan.json
```

Le rapport JSON est alors exploitable par d'autres étapes (diff de résultats, notification, tableau de bord).

14. Limites connues

1. **Heuristiques approximatives** — la longueur et la complexité des fonctions (JS/PHP/Java/C#/Go...) reposent sur un équilibre d'accolades : à confirmer par un humain.
1. **Contraintes de version « plage »** — une dépendance déclarée avec `>=` ou `^` est vérifiée par sa **version minimale** : une CVE introduite plus tard dans la plage peut être manquée.
1. **Secrets par entropie** — `secret-high-entropy` produit des résultats potentiellement faux : toujours vérifier avant suppression.
1. **Pas d'analyse sémantique multi-fichiers** — CodeScan analyse fichier par fichier : un flux de données traversant plusieurs fichiers (une entrée utilisateur « sale » utilisée loin de sa source) n'est pas suivi.
1. **Encodage** — les fichiers non-UTF-8 sont relus en latin-1 : l'analyse reste possible mais les extraits (`snippet`) peuvent être dégradés.

15. Mode IRON MAN AI — audit Kali (sites web)

L'outil de web audit s'appelle **IRON MAN AI** et s'utilise depuis une machine **Kali Linux**. Il pilote **tous les outils de sécurité disponibles** contre un site web et **réunit les failles** dans le même style de rapport (HTML /100, JSON ou PDF). C'est un **module séparé** de `main.py` : il ne modifie pas le comportement de l'analyse statique.

15.1 LA commande unique — audit complet en PDF

Une **seule commande** lance **tout l'audit ensemble** : tous les outils (web **et** invasifs), l'un après l'autre, **au maximum, sans limite de temps d'analyse**, avec toutes les autorisations requises, et produit

l'audit complet en PDF (plus JSON et HTML) :

```
uv run python ironman.py --url https://exemple.com --authorized
```

C'est l'équivalent de

`kali_scan.py --url ... --authorized --full --attack --pdf` : mode maximal (wordlist complète, `nmap -p- -sC`, threads élevés, **aucun timeout**, `sqlmap --level 3 --risk 3`) et rapport PDF 100 % stdlib (aucun navigateur requis). Utilisez `--tools` / `--exclude` / `--dry-run` pour restreindre, et `--check` d'abord pour vérifier que tout est installé.

15.2 Préflight (`--check`)

Vérifie la présence de chaque binaire (`nmap`, `nikto`, `whatweb`, `gobuster`, `dirsearch`, `ssllscan`, `nuclei`, `wafw00f`, `dnsrecon` ...).

Pour tout outil manquant, il affiche une commande

`sudo apt-get install -y <paquet>` exacte à copier.

```
uv run python kali_scan.py --check          # outils web présents ?
uv run python kali_scan.py --check --attack  # + sqlmap, xssstrike, commix, hydra
```

15.3 Scan normal (`kali_scan.py`)

Le flag `--authorized` est **obligatoire** (confirme que la cible est autorisée). Les outils **invasifs** sont exclus par défaut et ne tournent qu'avec `--attack` (hydra exige en plus les wordlists `--hydra-users` / `--hydra-passwords`).

```
uv run python kali_scan.py --url https://exemple.com --authorized \
  --output rapport_web.html
```

Options utiles : `--dry-run` (affiche les commandes sans exécuter),

`--tools nmap,nuclei` / `--exclude nmap` (filtrer), `--allow-missing`

(continuer malgré un outil manquant), `--keep-tmp` (conserver les logs

bruts par outil), `--pdf` (produire aussi le rapport PDF). Le **code de sortie** est identique au mode statique : 0 aucun relevé critique/haute,

1 sinon, 2 erreur d'usage.

Un **serveur local volontairement vulnérable** est fourni pour s'initier en toute sécurité :

```
uv run python examples/vuln_server.py --port 8123
uv run python kali_scan.py --url http://127.0.0.1:8123 --authorized \
  --output rapport_web.html --allow-missing
```

 **Manuel complet (Kali) :** [docs/MANUEL_KALI.md](#) · **Manuel Windows :**

[docs/MANUEL_WINDOWS.md](#) . Le manuel Kali se convertit en PDF via

[docs/make_pdf.py](#) (Chrome/Edge headless, sinon « Imprimer en PDF »).

Documentation générée pour IRON MAN AI (CodeScan) 1.4.0. Le README contient une version condensée ; ce manuel fait référence pour l'usage détaillé.

Généré par CodeScan — manuel imprimable.